

题意

Matrix53 和 Alex 都很喜欢下围棋。今天是中秋假期的第一天，Matrix53 邀请 Alex 来下围棋。举头望明月，Matrix53 突然想到，如果能知道自己当前的胜率该多好啊，这样就可以知道应该激进还是稳健行棋了。

Matrix53 把棋局抽象成了一棵树，树的每个结点代表一个局面，**树根代表初始局面**。Matrix53 或者 Alex 行棋，相当于选择了棋局所在结点的某个子结点。现在给定这棵树，以及棋局位于各个叶子结点时 Matrix53 的胜率，求棋局位于树根时 Matrix53 的胜率。

Matrix53 和 Alex 都会做出最优选择，且 Matrix53 先行棋。

题解

由题意，对于棋局抽象成的树，在每层节点上，有特定的决策对象，以根节点为第0层，则有偶数层上的决策者为Matrix，奇数层上的决策者为Alex。又由于两者都会根据另外一人的决策做出最优选择。故可以从树的最底层向上推导直至根节点。

由于结构体过于耗费时间且复杂，在这里用数组模拟树。我建立的主要的数组有4个，分别为：
parent, layer, rate, now

数组名称	作用	初始化操作	大小
<i>parent</i>	存储 <i>i</i> 节点的父节点	在读入时操作即可	N
<i>layer</i>	存储 <i>i</i> 节点所在层数	在读入时操作即可，同时可以找出树的最大层数 $maxlayer$	N
<i>rate</i>	存储 <i>i</i> 节点对应的最优决策的胜率	初始阶段时非叶节点（有子节点的节点）全部置-1，未置-1的结点赋叶节点的胜率值	N
<i>now</i>	两列数据结合索引 <i>flag</i> 交替记录当前的“叶节点”	<i>flag</i> 值置0， <i>now</i> 的第 <i>flag</i> 列赋所有对应叶节点，同时找出叶节点个数 <i>top</i>	$N * 2$

（ $N = 1e6 + 10$ ）

层数置为*maxlayer*

循环：

- 对*now*的*flag*列中当前层节点的父节点求胜率，而后将*now*中的当前叶节点置为其父节点，其他层结点不动。
 - 对当前的*now*的第*flag*列排序删去重复值（一个父节点拥有多个子节点）后填充至！*flag*列中，*flag*也置为！*flag*，同时*top*也置为新的值。
 - 当前层数减一，当层数为0时输出胜率。
- 排序时使用的为归并排序，时间复杂度为 $O\left(n * \log\left(n\right)\right)$ ，其他操作均为 $O\left(n\right)$

代码

```
#include <cstring>
#include <iostream>

using namespace std;

const int N = 1e6 + 10;
```

```

int parent[N], layer[N], rate[N], now[N][2], top = 0, flag = 0, tmp[N];

void merge_sort(int A[][2], int l, int r)
{
    if (l >= r)
        return;

    int mid = (l + r) >> 1;

    merge_sort(A, l, mid);
    merge_sort(A, mid + 1, r);

    int k = 0, i = l, j = mid + 1;
    while (i <= mid && j <= r)
        if (A[i][flag] <= A[j][flag])
        {
            tmp[k++] = A[i++][flag];
        }
        else
            tmp[k++] = A[j++][flag];
    while (i <= mid)
    {
        tmp[k++] = A[i++][flag];
    }
    while (j <= r)
        tmp[k++] = A[j++][flag];

    for (i = l, k = 0; i <= r; i++, k++)
        A[i][flag] = tmp[k];
}

int main()
{
#ifdef __LHY
    freopen("lhy.in", "r", stdin);
    freopen("lhy.out", "w", stdout);
#endif
    int n, x, y, r, i, maxlayer = 0;
    scanf("%d", &n);
    for (i = 1; i < n; i++)
    {
        scanf("%d%d", &x, &y);
        parent[y] = x;
        layer[y] = layer[x] + 1;
        rate[x] = -1;
        rate[y] = 0;
    }
    i = 1;
    while (scanf("%d", &r) != EOF)
    {
        while (rate[i] == -1)
            i++;
        rate[i] = r;
        now[top++][flag] = i;
        if (layer[i] > maxlayer)
            maxlayer = layer[i];
        i++;
    }
    // 按当前层循环
    for (int i = maxlayer; i > 0; i--)
    {

```

```

// 对now中当前层节点求父节点胜率
for (int j = 0; j < top; j++)
{
    if (layer[now[j][flag]] == i)
    {
        // matrix操作
        if (i % 2 == 1)
        {
            if (rate[now[j][flag]] > rate[parent[now[j][flag]]])
                rate[parent[now[j][flag]]] = rate[now[j][flag]];
            now[j][flag] = parent[now[j][flag]];
        }
        //alex操作
        else
        {
            if (rate[parent[now[j][flag]]] < 0)
                rate[parent[now[j][flag]]] = rate[now[j][flag]];
            if (rate[now[j][flag]] < rate[parent[now[j][flag]]])
                rate[parent[now[j][flag]]] = rate[now[j][flag]];
            now[j][flag] = parent[now[j][flag]];
        }
    }
}
//对now排序
merge_sort(now, 0, top - 1);
int tmp1 = -1, top_ = 0;
//剔除重复父节点并存至now的另一列中
for (int j = 0; j < top; j++)
{
    if (tmp1 == now[j][flag])
        continue;
    else
    {
        now[top_++][!flag] = now[j][flag];
        tmp1 = now[j][flag];
    }
}
top = top_;
flag = !flag;
}
printf("%d", rate[1]);
}

```